

It Is Not the Signature: An Empirical Study of a Hidden Key-Parse Cost in JSON Web Token Validation

Jannatul Ferdous Katha^{a,c}, Tasmia Tahmid Prova^{b,c}, Md Kishor Morol^c, Tze Hui Liew^{d,*}, Dip Nandi^{c,e}

^a*Department of CSE, Bangladesh University of Professionals (BUP), Dhaka, Bangladesh*

^b*Department of CSE, Bangladesh University of Business and Technology (BUBT), Dhaka, Bangladesh*

^c*ELITE Research Lab, Queens, NY, USA*

^d*Faculty of Information Science and Technology, Multimedia University, Melaka, Malaysia*

^e*Department of Computer Science, American International University-Bangladesh (AIUB), Dhaka, Bangladesh*

Abstract

Developers routinely attribute the per-request cost of JSON Web Token (JWT) validation to cryptography, and optimise or relocate it on that assumption. We test the assumption empirically in `jsonwebtoken`, a widely used Node.js library. The signature primitive is 2.208 μ s of a 29.92 μ s validation. The dominant component, 18.04 μ s (60%), is a discarded key-parse attempt: a shared secret supplied as a string is first offered to the asymmetric key parser and is resolved only after that parse fails. Its price is set by a transitive dependency, the OpenSSL release bundled with the runtime: issued from C, the same calls cost 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8. In a containerised service, validation adds 384.6 μ s of processor time per request, within 0.8% of the microbenchmark prediction. Of seven JWT libraries in four languages, only this one attempts the parse, so the cost follows a design choice rather than JWT validation as such. The obvious optimisation is unsafe: the parse entered the library with a key-confusion security fix, and removing it naively fails the library’s own tests. We give a narrow fix that passes the full test suite and makes validation 4.15 times faster, and identify pre-parsed keys

*Corresponding author.

Email addresses: `ferdous.katha@elitelab.ai` (Jannatul Ferdous Katha), `tasmia.prova@elitelab.ai` (Tasmia Tahmid Prova), `kishormorol@ieee.org` (Md Kishor Morol), `thliew@mmu.edu.my` (Tze Hui Liew), `dip.nandi@aiub.edu` (Dip Nandi)

as the remedy that removes the cost for every input. In our sample of 327 public call sites, none pre-parses the key. We derive guidance for developers, library authors and performance engineers, and release a replication package.

Keywords: Empirical study, performance analysis, software libraries, root-cause analysis, JSON Web Token, OpenSSL

1. Introduction

Performance engineering depends on attributing cost correctly. When a cost is believed to belong to one component and belongs to another, optimisation effort goes to the wrong place, and architectural remedies are adopted for problems that could have been fixed in a few lines. Empirical studies of performance bugs find that many stem from inefficient library usage and avoidable redundant work, and that the fixes are small once the cause is located (Jin et al., 2012; Selakovic and Pradel, 2016). This paper reports such a case on a path that runs on every request of many web services: the validation of JSON Web Tokens (JWTs) (Jones et al., 2015).

The usual reasoning is intuitive: validating a token means verifying a signature, signatures are cryptography, and cryptography is expensive, so the cost should be *moved* to a sidecar proxy, a gateway or dedicated hardware. The assumption has been tested before one layer down, where Coarfa et al. (2006) found RSA a minority of TLS server cost. We test it at the application layer, in `jsonwebtoken`, a widely used Node.js library, and find it wrong. A second natural explanation, that key material is converted from a string into a key object on every call, fails as well: the conversion costs $0.583\ \mu\text{s}$.

The actual mechanism is a control-flow accident (Figure 2). Given a secret that is not already a key object, the library attempts `createPublicKey()` first and falls back to `createSecretKey()` in the exception handler. For an HMAC secret the first call cannot succeed, so every validation constructs and discards an OpenSSL parse failure, at $18.04\ \mu\text{s}$ of a $29.92\ \mu\text{s}$ call. Its price depends on which OpenSSL release performs it, by more than an order of magnitude: runtimes carrying OpenSSL 3.2 or later pay tens of microseconds, runtimes carrying the 3.0 series pay hundreds. The 3.0 series is still the system OpenSSL of long-term-support distributions such as Ubuntu 22.04 and Debian 12, and of any image pinned to an older base layer, which is common in practice (Zerouali et al., 2019).

For software engineering the case matters for three reasons beyond the library itself. First, the cost sits in control flow that no profiler label names: a parse whose failure is caught and discarded appears in profiles as OpenSSL work, and the natural reading of that attribution is “cryptography”. Second, its magnitude is set by a transitive dependency that application developers do not choose directly and rarely record. Third, the obvious fix is unsafe. The resolution block entered the library in version 9.0.0, the release that fixed an RSA-to-HMAC key-confusion vulnerability (Auth0, 2022), and the key-type checks implementing that fix read the type of the key object it produces; removing it the obvious way fails the library’s own key-confusion tests. The code that costs the time is the code that makes a security check work, and the coupling is not visible from the lines being changed. The design that produces the cost (resolving key material on every call, by way of a parse whose failure serves as a type test) can also be named independently of `jsonwebtoken`, and it predicts which of seven libraries pay (Section 7).

Research questions.

RQ1 Where does the per-request cost of token validation actually go?

RQ2 What governs the magnitude of that cost?

RQ3 Does it matter in a running containerised service, and how common is the pattern that avoids it?

RQ4 Is the defect a property of one library, or of a design choice other libraries could share?

RQ5 Can the cost be removed without weakening the library’s key-confusion defence, and who can cause it to be paid?

Contributions. All are supported by measurements in the replication package (Katha et al., 2026).

1. Two independent decompositions of the validation call that locate its dominant component in a discarded key-parse attempt, and a split of that attempt into OpenSSL parse, V8 exception and binding (Section 4).
2. A causal attribution of its magnitude to the bundled OpenSSL release, with the parse reissued from C against 8 releases (Section 5).
3. Corroboration in a containerised service by cgroup CPU accounting, and a survey showing the avoiding pattern is rare in public code (Section 6).
4. A comparison of seven JWT libraries in four languages showing that the cost follows a design choice, not JWT validation as such (Section 7).

65 5. An analysis of why the cost cannot be removed naively, a narrow fix
66 that passes the library’s full test suite and is 4.15 times faster, and
67 practical guidance (Sections 8 and 9).

68 2. Background and related work

69 *What a validation call does.* A JSON Web Token carries a header, a payload
70 and a signature over the first two (Jones et al., 2015). Validating one means
71 decoding two segments, resolving key material into a form the cryptographic
72 library accepts, and verifying the signature. The middle step, key resolution,
73 is the subject of this paper, and it is the step most often left implicit when
74 validation cost is discussed.

75 *Where the cost is usually attributed.* Measurement of layered enforcement in
76 microservices concentrates on transport and proxy layers; Zhu et al. (2023)
77 show that service-mesh sidecar overhead depends on configuration and work-
78 load rather than being a fixed tax. Work on cryptographic library performance
79 has largely stayed at the level of the primitive or the protocol exchange (Naser
80 et al., 2020; Sikeridis et al., 2020). Our result concerns neither: it is the cost
81 of getting key material *into* the library, which we could not find measured
82 directly.

83 *Performance bugs.* Jin et al. (2012) found that most real-world performance
84 bugs follow a few recurring patterns, among them skippable function calls, and
85 that fixes are usually local; In JavaScript, Selakovic and Pradel (2016) identify
86 inefficient API usage as a leading root cause, and their later profiler (Selakovic
87 et al., 2017) targets the *order* in which a program evaluates conditions, the
88 axis on which this defect lives. The defect here is a skippable call, but one
89 whose failure is its only observable effect.

90 *Versions, dependencies and environment.* Performance changes across versions
91 in ways maintainers do not anticipate (Kaltenecker et al., 2023; Chen and
92 Shang, 2017). Container images pin a dependency set implicitly (Cito et al.,
93 2017), routinely ship outdated packages (Zerouali et al., 2019). For OpenSSL 3,
94 maintainers of a major downstream library have stated that parsing and key
95 loading regressed against 1.1.1 (Gaynor and Kehrler, 2026); Section 5.2 supplies
96 a controlled measurement for the one path this paper concerns.

97 *Measurement protocol.* Our protocol follows established guidance for managed-
98 runtime benchmarking (Georges et al., 2007; Laaber et al., 2019). The
99 attribution this paper overturns has the structure Mytkowicz et al. (2009) warn
100 about: a second variable, the bundled OpenSSL version, moving unrecorded
101 alongside the one under study.

102 *Cryptographic APIs and JWT libraries.* Most cryptographic vulnerabilities
103 are misuses of libraries by applications (Lazar et al., 2014; Egele et al., 2013),
104 largely because the interfaces are hard to use (Nadi et al., 2016); detection has
105 been automated by specifying correct usage (Krüger et al., 2018; Rahaman
106 et al., 2019). Our finding differs: the interface admits a use that is *correct*
107 and costly only for a reason invisible at the call site. Security analyses of
108 JWT implementations consistently find key-handling defects, key confusion
109 among them (Detering et al., 2017; Xu et al., 2023; Yang et al., 2026), and
110 RFC 8725 (Sheffer et al., 2020) codifies the resulting practice. The library
111 studied here has been hardened against key confusion twice: CVE-2015-9235,
112 fixed in 4.2.2 (GitHub Advisory Database, 2015), and CVE-2022-23541 with
113 two related defects, fixed in 9.0.0 (Auth0, 2022). We treat the same surface as
114 a cost site and show the two views are coupled. Our survey follows guidance
115 on mining public repositories (Kalliamvakou et al., 2014).

116 2.1. What we searched for and did not find

117 Table 1 summarises the positioning. Each cell records only what this
118 section states about the work cited; it is a map of the lines of work this paper
119 connects, not an assessment of them.

120 We found no peer-reviewed study of performance regressions across the
121 OpenSSL 3.x series, none tracing the chain from base image to bundled library
122 to application performance, and none treating a correct-but-costly use of a
123 cryptographic interface as a performance defect. We state each as an absence
124 we searched for.

125 3. Methodology

126 Figure 1 gives the study’s workflow: the instruments used for each research
127 question and the tables and figures each produces. Every instrument writes
128 to committed run files, from which every number in the paper is generated
129 (Data availability).

Table 1: Positioning against the closest lines of work. *JWT*: studies JWT or JOSE implementations. *Crypto-API security*: treats how a cryptographic interface or key material is used as a security concern. *Cost*: studies a performance cost or performance defect. *Version as cause*: attributes a cost or defect to the version of a bundled dependency. *Cost-defence coupling*: shows that removing a cost would weaken a security defence. ✓: addressed; ◦: partly (see note); -: not addressed.

Work	JWT	Crypto-API security	Cost	Version as cause	Cost-defence coupling
Detering et al. (2017); Xu et al. (2023); Yang et al. (2026)	✓	✓	–	–	–
Egele et al. (2013); Nadi et al. (2016); Krüger et al. (2018); Rahaman et al. (2019)	–	✓	–	–	–
Coarfa et al. (2006); Naser et al. (2020); Sikeridis et al. (2020)	–	–	✓	–	–
Zhu et al. (2023)	–	–	✓	–	–
Jin et al. (2012); Selakovic and Pradel (2016)	–	–	✓	–	–
Kaltenecker et al. (2023); Chen and Shang (2017)	–	–	✓	◦ ^a	–
Zerouali et al. (2019); Cito et al. (2017)	–	–	–	◦ ^b	–
This paper	✓	✓	✓	✓	✓

^aVersions of the system under study, not of a bundled dependency. ^bContainer images pin outdated dependency versions; no cost is measured.

130 3.1. Testbed

131 Unless stated otherwise, measurements were taken on one Apple-silicon
132 machine running macOS 26.5.2 (arm64), *the host*, under Node.js v26.6.0.
133 Container measurements ran in the Linux/arm64 virtual machine of Docker
134 Desktop’s single-node Kubernetes cluster (v1.32.2). The service studied in
135 Section 6 is a Node.js microservice (Express 5.1.0, `jsonwebtoken` 9.0.3) on
136 the `node:18-alpine` image, run as one replica with an Istio sidecar behind
137 an NGINX ingress, verifying HS256 tokens with a shared secret supplied as a
138 string. Load was generated in open loop by k6 v2.2.0 on the same machine, so
139 every in-situ window records the host load average observed during it. This
140 is a controlled single-node testbed, not a production deployment; we call it
141 the *containerised service*.

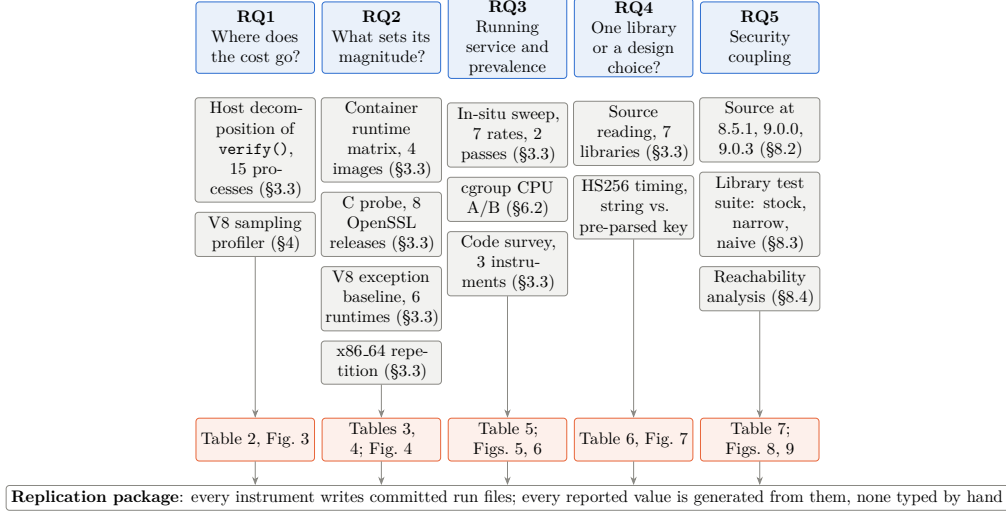


Figure 1: Study workflow. Top: the five research questions. Middle: the instruments that answer each, with the section describing it. Bottom: the tables and figures each produces. The C probe and the exception baseline (RQ2) also decompose the discarded attempt for RQ1 (Table 3), and the call-site survey (RQ3) supplies the algorithm-pinning figure used in RQ5. All measurements feed one replication package, from which every reported value is generated.

142 3.2. Measurement protocol

143 The unit of measurement is one operating-system process, not one call.
 144 Each process performs a fixed number of timed iterations of a single condition
 145 after a warmup of half that number and contributes its median; reported
 146 figures are medians of per-process medians, with percentile bootstrap intervals
 147 over processes (Georges et al., 2007). Conditions are interleaved within each
 148 round, and differences between conditions are reported as the median of per-
 149 round *paired* differences. The largest between-process coefficient of variation
 150 is 3.5% on the host.

151 3.3. Instruments

152 *Decomposing the call.* `jwt.verify()` of `jsonwebtoken` version 9.0.3 is de-
 153 composed into the operations it performs, each timed in isolation: structural
 154 decode, the HMAC primitive with a string and with a pre-parsed key, the
 155 symmetric key constructor, and the asymmetric key parse the library attempts
 156 first (Figure 2), both when it throws and when it succeeds. Each condition
 157 runs in 15 processes of 40000 calls.

```

120 if (secretOrPublicKey != null && !(secretOrPublicKey instanceof
    KeyObject)) {
121   try {
122     secretOrPublicKey = createPublicKey(secretOrPublicKey);
123   } catch (_) {
124     try {
125       secretOrPublicKey = createSecretKey(typeof
        secretOrPublicKey === 'string' ? Buffer.from(
        secretOrPublicKey) : secretOrPublicKey);
126     } catch (_) {
127       return done(new JwtTokenError('secretOrPublicKey is not
        valid key material'))
128     }
129   }
130 }

```

Figure 2: Key resolution in `jsonwebtoken` version 9.0.3, `verify.js`, lines 120--130. Verbatim but for the common leading indentation, which is removed to fit the column; line numbers are the file's own. A secret that is not already a `KeyObject` is offered to `createPublicKey()` first, and `createSecretKey()` is reached only from the exception handler. For an HMAC secret supplied as a string the outer `try` can never succeed, so the asymmetric parse is attempted and discarded on every validation.

158 *Runtime matrix.* Containerisation is held constant and the runtime varied:
 159 four container images spanning two OpenSSL series and three V8 major
 160 versions, on one machine, at 10 processes of 20000 calls. The runtime identity
 161 of each image is recorded with every row and cross-checked (0 mismatches).

162 *C probe.* To remove the JavaScript engine, we reproduced in C, call for call
 163 from the Node source, what `createPublicKey()` asks of OpenSSL for an
 164 HMAC secret (three PEM scans, a private-key parse and the error-queue
 165 drain), compiled with one compiler (GCC 13.3.0) against 8 OpenSSL releases
 166 from 3.0.16 to 3.6.1 built identically from source, at 15 processes per release.

167 *Exception baseline.* An `Error` of the same message, properties and code as
 168 Node's (captured from a real failure) is thrown from 8 frames and discarded,
 169 with no OpenSSL call, on six official Node runtimes. With the C probe this
 170 gives a three-way decomposition of the discarded attempt: the OpenSSL
 171 parse, the V8 exception, and a remainder attributed to the binding between
 172 them.

173 *Other libraries.* We read the key-handling code of six further libraries,
 174 the most used in their ecosystems (`fast-jwt`, `jose`, `PyJWT`, `golang-jwt`,
 175 `nimbus-jose-jwt`, `jjwt`), and timed HS256 verification in each with the
 176 secret as a string (or the rawest accepted form) and as the library’s own
 177 pre-parsed key object, at 15 processes of 20000 calls.

178 *Second architecture.* The host decomposition, the runtime matrix, the C
 179 probe, the exception baseline and the cross-library comparison were measured
 180 on an x86_64 virtual machine (Intel(R) Xeon(R) Processor @ 2.80GHz, four
 181 vCPUs) on shared cloud infrastructure. Following Laaber et al. (2019), we
 182 take only orderings and ratios between interleaved conditions from it.

183 *In-situ measurement.* The containerised service exposes a histogram timing
 184 the verification call alone, read before and after each of seven arrival-rate
 185 steps; means are exact differences of `_sum` over `_count`. The sweep ran twice
 186 on one process, ascending then descending. An A/B at one rate, validation
 187 enabled and then disabled, reads the processor time charged to the container
 188 from cgroup accounting.

189 *Prevalence.* Three instruments with different frames, captured on 2026-09-18:
 190 code-search co-occurrence counts, a classified convenience sample of call sites
 191 (one file per repository, key argument traced within its file), and a targeted
 192 counter-search that fetches every file co-occurring the library with the pre-
 193 parsing interface and adjudicates it by hand. None supports a population
 194 proportion, so results are reported *in our sample*.

195 4. RQ1: Where the cost goes

196 Table 2 and Figure 3 give each operation in the validation path. The
 197 signature primitive costs 2.208 μ s of a 29.92 μ s call. The conversion to which
 198 the remainder is most readily attributed, the symmetric key constructor, costs
 199 0.583 μ s. The discarded attempt that precedes it costs 18.04 μ s, 60% of the
 200 call and its largest single component. Supplying a pre-parsed key instead of a
 201 string reduces the call by 23.92 μ s [23.58, 24.00]. The asymmetry often read
 202 as conversion being dearer than parsing is backwards: a *successful* asymmetric
 203 parse costs 13.83 μ s, so the RSA path’s parse is useful work while the HMAC
 204 path’s is waste.

Table 2: Cost of each operation in the validation path. Medians of per-process medians over 15 processes of 40000 calls; 95% percentile bootstrap intervals over processes, which with 15 units are indicative rather than calibrated (Section 3.2).

Operation	Median (μs)	95% interval (μs)
<code>verify()</code> , string secret	29.92	[29.75, 30.12]
discarded asym. parse	18.04	[17.83, 18.25]
successful asym. parse	13.83	[13.62, 13.92]
<code>verify()</code> , pre-parsed key	6.042	[5.958, 6.125]
HMAC, string key	2.208	[2.208, 2.250]
HMAC, pre-parsed key	2.166	[2.125, 2.167]
structural decode	1.292	[1.250, 1.292]
symmetric key constructor	0.583	[0.542, 0.583]
timer overhead	0.042	[0.042, 0.042]

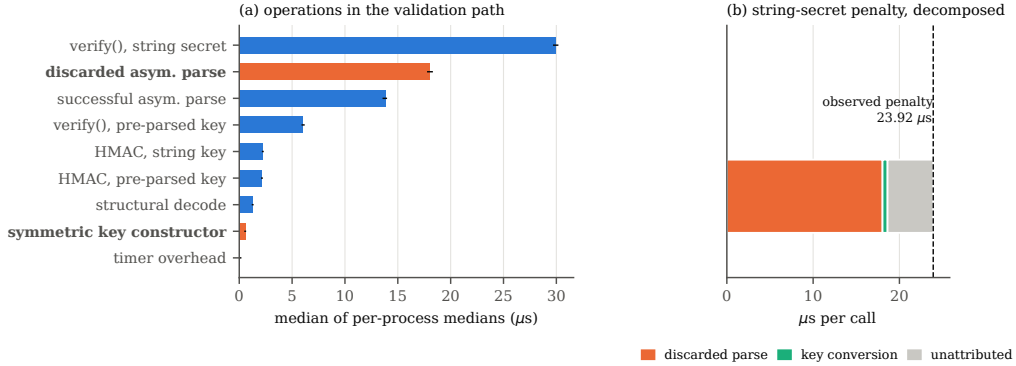


Figure 3: (a) Every operation in Table 2, on a linear scale, with the two components the paper turns on emphasised; whiskers are the 95% percentile bootstrap intervals over 15 processes. (b) The 23.92 μs penalty for supplying a string secret rather than a pre-parsed key, decomposed. The three segments are medians of per-round paired differences, not a partition of a single measurement, and they do not sum to the penalty: the 5.292 μs remainder is unattributed (Section 4.2).

205 4.1. What the discarded attempt is made of

206 Table 3 decomposes the attempt on six official runtimes. On every runtime
 207 bundling OpenSSL 3.0 the C parse alone matches the probe to within a few
 208 percent (101% of it under Node 18.20.8): the discarded attempt *is* the

Table 3: The discarded attempt decomposed, per runtime, in μs (x86_64). *Probe*: `createPublicKey(secret)` inside Node, thrown and caught. *C parse*: the same OpenSSL calls issued from C against the OpenSSL release that runtime bundles, built from source. *V8 exception*: an `Error` of the same shape, message and properties thrown from 8 frames and discarded, with no OpenSSL call. *Binding*: the probe less the other two; its interval, from independent resampling of the three runs, is in the replication package. Medians of per-process medians over 15 processes (probe, exception) and 15 processes (C). The C column uses a stock build of each release, not Node’s own build of it, so the binding column is approximate (Section 10).

Runtime	OpenSSL	Probe	C parse	V8 exception	Binding
Node 18.20.8	3.0.16	390.5	394.4	4.781	-8.694
Node 20.20.2	3.0.19	443.4	431.3	4.796	7.292
Node 23.11.1	3.0.16	383.5	394.4	3.959	-14.88
Node 22.23.3	3.5.8	18.98	8.447	4.021	6.515
Node 24.21.0	3.5.8	19.54	8.447	4.136	6.952
Node 26.10.0	3.5.8	18.01	8.447	3.421	6.145

209 OpenSSL parse and would cost the same with no JavaScript engine. The V8
 210 exception is small and nearly constant, about 4.781 μs (1.2% of the probe
 211 under Node 18.20.8), half of it stack capture. Where OpenSSL 3.5 makes the
 212 parse cheap, the three parts are comparable: under Node 26.10.0 the parse
 213 is 47%, the exception 19.0% and the binding 34.1%. The component is a
 214 discarded *parse*, not an exception cost.

215 4.2. Checks on the decomposition

216 The discarded parse and the conversion account for 77.9% of the string
 217 penalty; the 5.292 μs residual [4.916, 5.501] lies in the library’s type and claim
 218 checking and is reported as unattributed. Because the decomposition relies
 219 on differencing, it was checked against V8’s sampling profiler: the asymmetric
 220 key parse accounts for a median 66.2% of self time on the host, 95.3% under
 221 Node 18 and 61.5% under Node 26, and does not appear among the top
 222 frames of the pre-parsed path. Two methods, one answer.

223 5. RQ2: What governs the magnitude

224 5.1. What the runtime matrix shows

225 Table 4 varies the runtime with containerisation held constant. The dis-
 226 carded parse spans $25.36\times$ across the four images, while pre-parsed validation

Table 4: The same measurement across four container runtimes on one machine, 10 processes each of 20000 calls. The *Disc. parse*: the discarded asymmetric parse; *Pre-parsed*: `verify()` with a pre-parsed key. [‡]The host row is from Table 2 and is not containerised. [†]Inferred from the run timestamp, not recorded per row: that run predates the per-measurement OpenSSL field (Section 10). The discarded parse spans $25.36\times$ across the four images while pre-parsed validation spans $1.26\times$.

Runtime	OpenSSL	V8	Disc. parse (μs)	Pre-parsed (μs)
Node 18 musl	3.0.16	10	373.79	7.58
Node 20 musl	3.0.19	11	420.62	7.25
Node 26 musl	3.5.7	14	22.94	6.56
Node 26 glibc	3.5.7	14	16.58	6.04
host [‡]	3.6.3 [†]	14	18.04	6.042

spans only $1.26\times$. Node 18 and Node 20 ship different V8 major versions and share the OpenSSL 3.0 series; both pay of order 373.79–420.62 μs , so within that series the engine does not move the cost. A successful parse moves with the failing one ($11.74\times$), so the effect concerns key parsing in general. The matrix alone cannot separate OpenSSL from the engine across the series boundary, since the fast images also carry a newer engine and a different build; the crossed designs below do.

5.2. Caused by OpenSSL

In plain C, with nothing else varying, the failure costs 394.4 μs on OpenSSL 3.0.16 and 8.447 μs on 3.5.8, a ratio of 46.69 [46.16, 46.90] (Figure 4). The magnitude is therefore *caused* by the OpenSSL release, for this code path. Almost all of it is the private-key parse (99% on 3.0.16), where OpenSSL 3 hands the input to its provider-based decoders. The cost fell in two steps, to 57.48 μs in 3.1.8 and to 6.998 μs in 3.2.6, and is flat thereafter. The engine is excluded from the other direction too: Node 23.11.1 pairs a *newer* V8 with the *older* OpenSSL 3.0.16 and pays 383.5 μs , against 18.98 μs for Node 22.23.3 with OpenSSL 3.5.8.

A second architecture. On x86_64 every OpenSSL 3.0 image is again slower than every OpenSSL 3.5 image (the split holds): the discarded parse spans $30.50\times$ across the images while pre-parsed validation spans $1.31\times$. The host decomposition keeps its ordering: the discarded parse is 19.42 μs of a 36.48 μs call (53%).

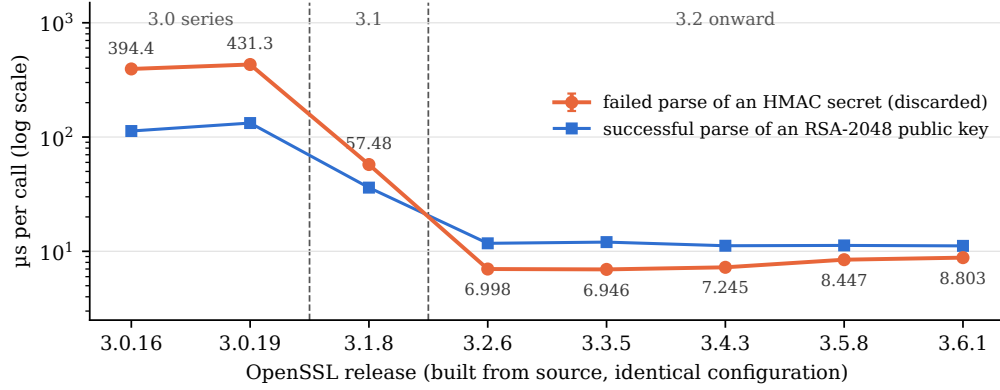


Figure 4: The failed parse of an HMAC secret issued from C, with no JavaScript engine, against 8 OpenSSL releases built from source with one configuration, beside a successful parse of an RSA-2048 public key through the same path. Whiskers are 95% percentile bootstrap intervals over 15 processes and are narrower than the markers. The cost falls in two steps, at 3.1 and 3.2, and is flat thereafter.

6. RQ3: The containerised service, and prevalence

6.1. In-situ cost

Figure 5 shows the in-situ cost at seven arrival rates. The cost is present in the containerised service and far larger than the host figure, consistent with Section 5: the service’s runtime carries the older OpenSSL, whose discarded parse costs 373.79 μ s, giving an expected total near 381 μ s. Wall-clock means of 453–1028 μ s exceed that; contention inflates durations, so they are upper bounds. We do not interpret the decline with arrival rate or the systematic gap between the two passes, which have several candidate causes unrelated to validation.

6.2. Corroboration: the cost is processor time

To separate work from waiting, Table 5 compares the same service with validation enabled and disabled, reading processor time from cgroup accounting rather than from a clock. Enabling validation adds 384.6 μ s of processor time per call, within 0.8% of the 381 μ s the runtime matrix predicts, from instruments that share no code. A request that queues accrues wall-clock time but no CPU, so this figure contains no queueing delay. The sidecar’s CPU did not rise with the application’s, and the one-second CPU samples

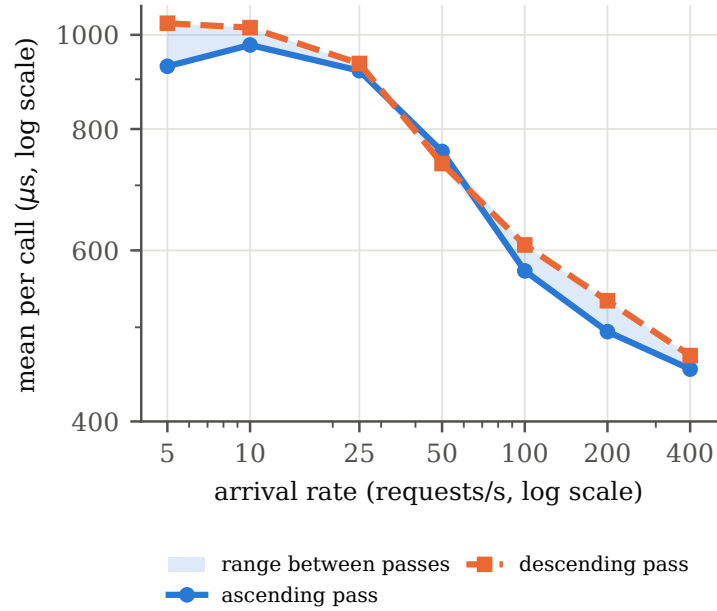


Figure 5: In-situ validation cost against arrival rate, string secret. The two passes are drawn separately rather than averaged, because they differ systematically and averaging would conceal that. The shaded band is the range between the passes, which is two observations rather than an interval.

Table 5: Validation enabled against validation disabled, at one arrival rate, two 180 s windows on the same pod and image, 36001 requests each. A signed token is on the wire in both arms; the disabled arm removes only the handler’s `jwt.verify()` call. Application and sidecar CPU are means of one-second cgroup samples in the window; CPU per call is that mean divided by the achieved rate. Event-loop lag is the mean of the per-scrape p99. Host load is a single sample at window start.

	disabled	enabled
Achieved rate (req/s)	200.006	200.006
Application CPU (millicores)	69.45	146.38
CPU per call (μs)	347.3	731.9
Sidecar CPU (millicores)	53.40	49.13
Event-loop lag (ms)	3.137	3.159
Client mean latency (ms)	1.6802	2.0332
Host load, 1 min	3.53	5.11

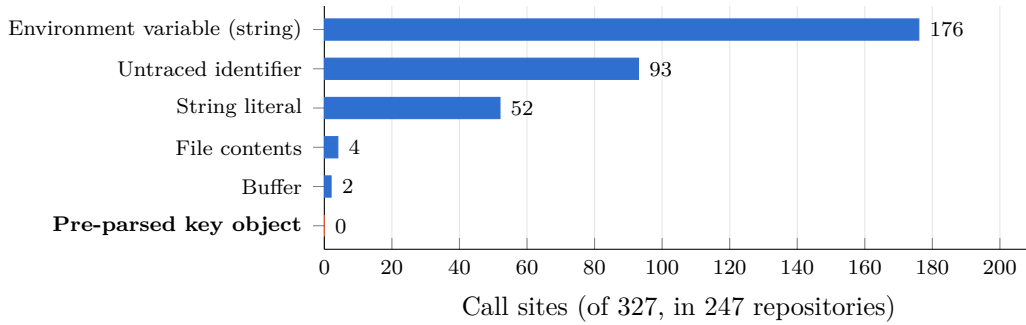


Figure 6: Where the secret passed to the validator comes from, for the 327 call sites classified in our sample. No call site passes a pre-parsed key object, the one form that avoids the discarded parse for every input. Independently, 309 of the 327 call sites (94%) pass no algorithm restriction. Counts describe the sample, not the population (Section 3.3).

of the two arms do not overlap (disabled never exceeded 100.577 millicores, enabled never fell below 114.963).

6.3. How common is the avoiding pattern, in our sample?

In code-search counts, no cell exceeds 0.0303% of files importing the library that also mention the pre-parsing interface. Of 327 call sites classified across 247 repositories, 0 pass a pre-parsed key (Figure 6), and 309 (94%) pass no algorithm restriction. Call sites are found by resolving the local binding of the import, which adds 12 that the conventional `jwt.verify()` spelling misses. The targeted counter-search fetched 191 files, 120 with a validation call in 112 repositories; of 13 call sites flagged automatically, 5 were false positives on hand inspection and 7 genuine, in 6 projects. Even among repositories that know the interface exists, only 5.36% use it: the pattern is rare, not absent.

7. RQ4: One library, or a design choice?

A cost found in one library becomes a statement about a class of defect only if its mechanism can be named independently and predicts which other libraries have it. The mechanism here is *resolving key material of unknown type by attempting the asymmetric parse first, and treating its failure as the signal that the material is a shared secret*. Table 6 and Figure 7 test that prediction.

Table 6: HS256 verification across seven JWT libraries in four languages (x86_64), in μ s. *Asym. first?*: given a string HMAC secret, does the library attempt to parse it as an asymmetric key, as read from its source. *String* and *pre-parsed*: the secret in the form the library’s documentation shows, and in the library’s own key object built once (Section 3.3). *Penalty*: median of per-round paired differences, with a 95% bootstrap interval over rounds. Medians of per-process medians over 15 processes of 20000 calls. Absolute costs measure different amounts of work in different runtimes and are not a ranking of libraries. ^aNo string accepted for an HMAC key; the rawest accepted form (bytes) is used. ^bA string is accepted only by a deprecated setter that resolves it once, when the parser is built.

Library	Lang.	Version	Crypto backend	Asym. first?	String	Pre-parsed	Penalty	95% interval
jsonwebtoken	JS	9.0.3	OpenSSL 3.0.19	yes	476.2	10.23	465.9	[465.2, 467.8]
jsonwebtoken	JS	9.0.3	OpenSSL 3.5.8	yes	34.22	9.844	24.56	[24.07, 24.82]
fast-jwt	JS	6.3.3	OpenSSL 3.0.19	no	7.564	7.570	0.070	[-0.116, 0.111]
fast-jwt	JS	6.3.3	OpenSSL 3.5.8	no	7.917	7.934	-0.068	[-0.134, 0.197]
jose	JS	6.2.12	OpenSSL 3.0.19	no ^a	121.2	78.42	42.66	[38.50, 45.89]
jose	JS	6.2.12	OpenSSL 3.5.8	no ^a	98.89	73.98	24.80	[21.91, 26.51]
PyJWT	Python	2.15.0	hashlib, OpenSSL 3.0.13	no	41.33	41.28	-0.103	[-0.457, 0.338]
golang-jwt	Go	5.3.1	Go crypto/hmac	no ^a	4.810	4.764	0.034	[-0.042, 0.117]
nimbus-jose-jwt	Java	10.10	JCA SunJCE	no	4.358	4.229	0.304	[-0.098, 1.016]
jjwt	Java	0.13.0	JCA SunJCE	no ^b	12.15	12.33	-0.406	[-0.715, 0.648]

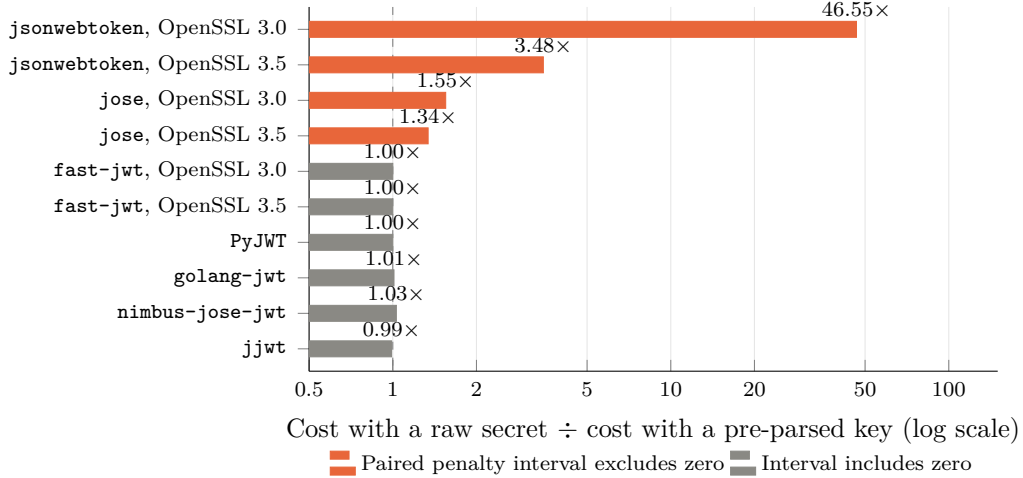


Figure 7: How much each library pays for being handed its raw secret rather than its own pre-parsed key object: the ratio of the two costs in Table 6, on a log scale. A ratio of one (dashed line) means no per-call penalty. Orange: the paired 95% interval on the penalty excludes zero. Only `jsonwebtoken` performs the failing asymmetric parse, and only it pays by more than a small factor; `jose` pays for re-importing raw bytes on every call.

286 Only `jsonwebtoken` attempts the asymmetric parse. The others avoid it
287 in one of three ways: *inspect, then dispatch* (`fast-jwt` and `PyJWT` examine the
288 key text and call exactly one constructor); *refuse the ambiguous input* (`jose`
289 and `golang-jwt` do not accept a string HMAC key); or *type the key* (`jjwt`
290 requires a `SecretKey`; `nimbus-jose-jwt` converts a string’s bytes without
291 parsing them). Of the seven libraries, two pay for a raw secret, in the sense
292 that the paired interval on the penalty excludes zero. `jsonwebtoken` pays
293 465.9 μ s under OpenSSL 3.0.19 and 24.56 μ s under OpenSSL 3.5.8; `jose`,
294 which re-imports raw bytes into a `WebCrypto` key on every call, pays about
295 11 times less. The other five pay nothing measurable. All seven validate the
296 same kind of token with the same primitive, so the cost follows a design choice,
297 resolving key material on every call, and the failing parse is its costliest form.
298 That is the class of order-of-evaluation defect the profiler of Selakovic et al.
299 (2017) targets.

300 8. RQ5: Removing the cost safely

301 8.1. Threat model, and the attempt precedes authentication

302 The adversary is a remote client that can place arbitrary bytes where the
303 service expects a bearer token, including every header field, but does not
304 know the shared secret. The asset is key-type separation: a token declaring
305 an HMAC algorithm must never be verified against asymmetric key material,
306 nor the reverse (Sheffer et al., 2020). Availability is out of scope; we drove no
307 adversarial traffic. Figure 8 shows the order of checks in `verify()`. Reaching
308 key resolution requires only a well-formed token; the permitted-algorithm list,
309 the key-type checks and the signature all come later. A token that will be
310 rejected therefore pays the discarded attempt in full, and restricting permitted
311 algorithms does not avoid it.

312 8.2. The attempt arrived with a key-confusion fix

313 Comparing `verify.js` in versions 8.5.1, 9.0.0 and 9.0.3: in 8.5.1 the
314 secret is passed to the signature routine as given, and the resolution block of
315 Figure 2 first appears in 9.0.0, the release that fixed CVE-2022-23541 and
316 CVE-2022-23539 (Auth0, 2022). The checks implementing those fixes (lines
317 148–160) operate on the key object the block produces. The attempt is part
318 of a security fix, not an oversight.

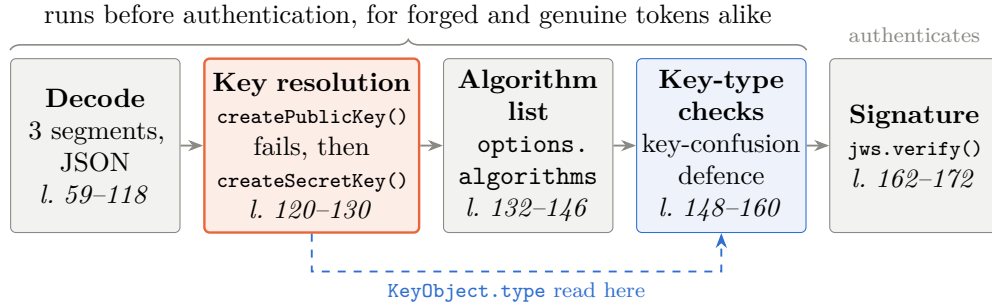


Figure 8: Order of operations in `verify()` at tag v9.0.3, with the file’s line numbers. Every step up to and including the algorithm and key-type checks runs before the signature is verified, and so runs for a forged token exactly as for a genuine one. The discarded attempt sits inside key resolution; the key-type checks that implement the library’s key-confusion defence read the type of the key object resolution produces (dashed arrow). Pinning permitted algorithms is enforced at the third step and therefore does not prevent the second.

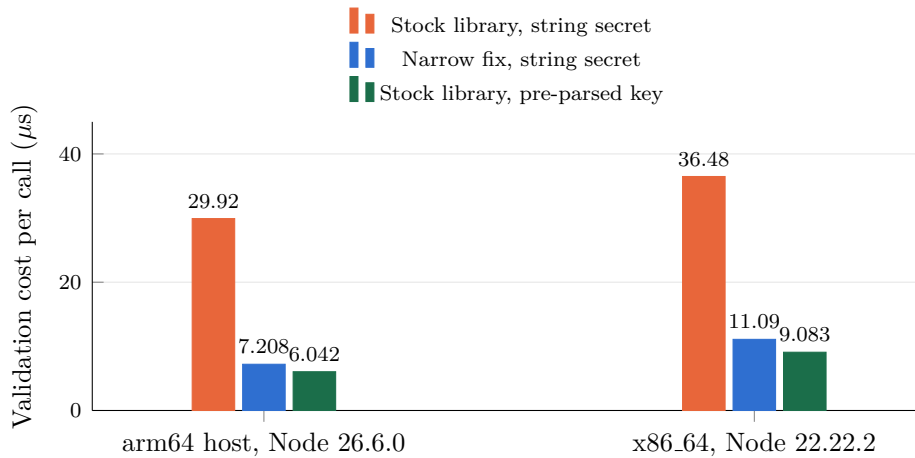


Figure 9: Per-call HS256 validation cost under the stock library with a string secret, under the narrow fix with the same string secret, and under the stock library with a pre-parsed key, on both machines (medians of per-process medians). The narrow fix recovers most of the gap without any change to calling code; only the pre-parsed key removes the attempt for every input (Section 8.4).

319 8.3. A fix, and a constraint on it

320 The fix dispatches on the token’s declared algorithm before attempting
 321 the asymmetric parse, so an HMAC token with an ordinary string secret is

Table 7: Which tokens reach the discarded key-parse attempt, for a service configured with an HMAC shared secret, as read from the source at tag v9.0.3 and from the patch of Section 8.3. “Well-formed”: three segments, JSON header and payload, non-empty signature. No row requires knowledge of the secret.

Configuration	Tokens that reach the parse
Stock, string secret	every well-formed token, forged or genuine
Narrow fix, plain string secret (no PEM or JWK form)	every well-formed token whose header declares a non-HMAC algorithm; no such token can be valid for this service
Pre-parsed <code>KeyObject</code>	none

322 resolved directly. It takes the call from 29.92 μ s to 7.208 μ s, a factor of 4.15,
 323 and on x86.64 from 36.48 μ s to 11.09 μ s (Figure 9), with no change in calling
 324 code. Against the library’s own suite at tag v9.0.3 the patch leaves results
 325 unchanged: 511 passing, 0 failing.

326 The constraint is the more important half. The *success* of the parse
 327 on asymmetric key material produces the key object whose type the key-
 328 confusion check depends on. The fix must therefore be narrow: the fast
 329 path is restricted to string secrets that cannot be asymmetric key material.
 330 Removing that restriction, the obvious optimisation, produces 2 failures in
 331 the library’s key-confusion tests (“should not allow HMAC verification with
 332 an RSA key in PEM format” and, under “signing with pub key as symmetric”,
 333 “should not verify”). The existing suite already guards the property; what is
 334 missing is visibility of the coupling to someone changing the code.

335 8.4. What the narrow fix leaves reachable

336 The fast path is selected by the declared algorithm, which the token’s
 337 author chooses. A token declaring a non-HMAC algorithm falls through to
 338 the original resolution and pays the parse before being rejected (Table 7).
 339 The narrow fix removes the cost from legitimate traffic, not from traffic an
 340 adversary constructs. Only a pre-parsed key object removes it for every input,
 341 because resolution is skipped when the key is already a `KeyObject`.

342 8.5. Reported upstream

343 An earlier issue¹ reports HS256 verification in 9.x 34 times slower than in
344 8.5.1 on OpenSSL 3.0, with no diagnosed cause. We reported the mechanism²,
345 and a pull request implementing the narrow fix is open. At the time of writing
346 neither has received a maintainer response.

347 9. Implications for practice

348 Table 8 maps each finding to its evidence and to the stakeholder who can
349 act on it.

350 *Application developers.* Pre-parse key material once, at startup. It removes
351 23.92 μ s per validation on the host and far more on OpenSSL 3.0, needs no
352 architectural change, and is the only remedy that removes the attempt for
353 every input. Also restrict permitted algorithms (Sheffer et al., 2020); that is
354 an independent defence, not a substitute.

355 *Performance engineers.* Treat the base image as part of the measurement.
356 The same code differs by $25.36\times$ on this path across images because they
357 pin different OpenSSL releases, and a regression after a base-image bump
358 may have nothing to do with the application. Profile before relocating a cost
359 believed to be cryptographic to a sidecar or gateway.

360 *Library authors.* Do not use a failing parse as a type test. Inspect and
361 dispatch, refuse ambiguous input, or type the key, and resolve keys once
362 rather than on every call.

363 *Maintainers and reviewers.* Review performance changes to verification code
364 as security changes, run them against the key-confusion tests, and time them
365 on the oldest cryptographic library a deployment may bundle.

366 10. Threats to validity

367 *Construct.* The decomposition relies on differencing, cross-checked by a pro-
368 filer. The C probe reproduces Node’s OpenSSL calls but not Node’s own
369 build of OpenSSL, so the binding column of Table 3 is approximate; the

¹<https://github.com/auth0/node-jsonwebtoken/issues/966>

²<https://github.com/auth0/node-jsonwebtoken/issues/1046>

Table 8: Summary of findings, evidence and implications, by research question and by the stakeholder who can act on each.

RQ	Finding	Evidence	Implication (who acts)
RQ1	The signature is 2.208 μ s of a 29.92 μ s call; a discarded key-parse attempt is 18.04 μ s (60%).	Table 2, Fig. 3; profiler	Profile before relocating a “cryptographic” cost; attribution by intuition was wrong twice here (<i>performance engineers</i>).
RQ2	The parse costs 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8, issued from C.	Table 4; Fig. 4	Record the base image and bundled OpenSSL with every performance figure; treat base-image bumps as performance changes (<i>developers, DevOps</i>).
RQ3	In a containerised service validation adds 384.6 μ s of CPU per request; no sampled call site pre-parses its key.	Table 5; Figs. 5, 6	Pre-parse key material once at startup: a one-line change (<i>application developers</i>).
RQ4	Of seven libraries in four languages only one attempts the parse; five pay nothing for a raw secret.	Table 6, Fig. 7	Do not use a failing parse as a type test; resolve keys once, inspect then dispatch, or type the key (<i>library authors</i>).
RQ5	The parse is load-bearing for a key-confusion defence; the naive optimisation fails 2 library tests, the narrow fix passes all 511 and is 4.15 $\times$ faster.	Table 7; Figs. 8, 9	Review performance changes to verification code as security changes; run the key-confusion tests (<i>maintainers, reviewers</i>).

causal claim compares releases under one build and does not inherit this. The security analysis is from source and the library’s tests, not from adversarial traffic.

Internal. The in-situ evidence is one process sweeping seven rates twice, and the A/B is two consecutive windows; the magnitude claim rests on CPU accounting, which contention cannot inflate. The headline host figure predates the per-measurement OpenSSL field; its version is inferred from the run record.

378 *External.* Primary measurements come from one ARM64 host; the x86_64
379 machine is a shared virtual machine, used only for orderings and ratios. The
380 service is a single-node testbed, and a service caching verified tokens pays the
381 cost once per token. Seven libraries at one version each show a design choice,
382 not how often it is made. The survey resolves call sites within one file and
383 was adjudicated by one rater; verdicts and file hashes are published.

384 *Future work.* Driving forged tokens at the service would turn reachability into
385 an availability measurement; bisecting OpenSSL between 3.0 and 3.2 would
386 identify the responsible changes; and a systematic survey of JWT libraries
387 would estimate how often the design choice is made.

388 11. Conclusion

389 The per-request cost of token validation in `jsonwebtoken` is not the
390 signature (2.208 μ s of 29.92 μ s) and not key conversion (0.583 μ s). It is a
391 parse attempted in the wrong order and discarded on every request, 60%
392 of the call. Its price is set causally by the OpenSSL release a base image
393 pins: 394.4 μ s on 3.0.16 against 8.447 μ s on 3.5.8. It is a design choice, not a
394 property of JWT, and it cannot be removed naively because it is load-bearing
395 for a key-confusion defence. A narrow fix preserves the defence and is 4.15
396 times faster, and pre-parsing the key once is the only configuration no input
397 can push onto the slow path, yet no call site in our sample does so.

398 The broader lesson is about attribution. The explanation that presented
399 itself before measurement was wrong twice, and the component that sets
400 the cost is one that measurement protocols rarely record. Performance
401 studies should record the identity of the components they measure, including
402 transitive dependencies such as the bundled cryptographic library, at the
403 granularity at which they can change.

404 CRedit authorship contribution statement

405 **Jannatul Ferdous Katha:** Conceptualization, Methodology, Software,
406 Investigation, Data curation, Formal analysis, Visualization, Writing – original
407 draft. **Tasmia Tahmid Prova:** Validation, Writing – review & editing. **Md**
408 **Kishor Morol:** Supervision, Writing – review & editing. **Tze Hui Liew:**
409 Supervision, Project administration, Writing – review & editing. **Dip Nandi:**
410 Supervision, Writing – review & editing.

411 **Declaration of competing interest**

412 The authors declare that they have no known competing financial interests
413 or personal relationships that could have appeared to influence the work
414 reported in this paper.

415 **Funding**

416 This research did not receive any specific grant from funding agencies in
417 the public, commercial, or not-for-profit sectors.

418 **Data availability**

419 The data supporting the results of this study are openly available in the
420 replication package (Katha et al., 2026), archived at Zenodo under <https://doi.org/10.5281/zenodo.23011153> and maintained in a public GitHub
421 repository.³ It contains all measurements, code and scripts that generate
422 every value and figure in this paper. The third-party survey corpus is not
423 redistributed; a manifest pins each of its 502 files by content hash.
424

425 **Ethics and responsible disclosure**

426 The study involves no human participants and no personal data, and
427 no attack was run against any system the authors do not control. The key-
428 confusion behaviour discussed is a publicly documented attack class, exercised
429 only through the library’s own test suite.

430 **Declaration of generative AI and AI-assisted technologies in the** 431 **manuscript preparation process**

432 During the preparation of this manuscript, generative AI tools were used to
433 improve language and readability. The authors reviewed all content following
434 such use and take full responsibility for the contents of the manuscript.

³<https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing->

435 Acknowledgements

436 The authors acknowledge ELITE Research Lab for providing the compu-
437 tational resources and support necessary for this research.

438 References

- 439 Auth0, 2022. CVE-2022-23539, CVE-2022-23541, CVE-2022-23540: Security update
440 for jsonwebtoken. Auth0 security bulletin, 21 December 2022, [https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-j](https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jsonwebtoken)
441 [sonwebtoken](https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jsonwebtoken). Accessed 2026-09-28.
- 443 Chen, J., Shang, W., 2017. An exploratory study of performance regression
444 introducing code changes, in: 2017 IEEE International Conference on Software
445 Maintenance and Evolution (ICSME). doi:10.1109/ICSME.2017.13.
- 446 Cito, J., Schermann, G., Wittern, J.E., Leitner, P., Zumberi, S., Gall, H.C., 2017.
447 An empirical analysis of the Docker container ecosystem on GitHub, in: 2017
448 IEEE/ACM 14th International Conference on Mining Software Repositories
449 (MSR). doi:10.1109/MSR.2017.67.
- 450 Coarfa, C., Druschel, P., Wallach, D.S., 2006. Performance analysis of TLS web
451 servers. ACM Transactions on Computer Systems 24. doi:10.1145/1124153.11
452 24155.
- 453 Detering, D., Somorovsky, J., Mainka, C., Mladenov, V., Schwenk, J., 2017. On
454 the (in-)security of JavaScript object signing and encryption, in: Proceedings
455 of the 1st Reversing and Offensive-oriented Trends Symposium (ROOTS '17).
456 doi:10.1145/3150376.3150379.
- 457 Egele, M., Brumley, D., Fratantonio, Y., Kruegel, C., 2013. An empirical study
458 of cryptographic misuse in Android applications, in: Proceedings of the 2013
459 ACM SIGSAC Conference on Computer and Communications Security (CCS
460 '13). doi:10.1145/2508859.2516693.
- 461 Gaynor, A., Kehr, P., 2026. The state of OpenSSL for pyca/cryptography.
462 pyca/cryptography project statement, [https://cryptography.io/en/late](https://cryptography.io/en/latest/statements/state-of-openssl/)
463 [st/statements/state-of-openssl/](https://cryptography.io/en/latest/statements/state-of-openssl/). Maintainer statement, not peer reviewed;
464 accessed 2026-09-24.
- 465 Georges, A., Buytaert, D., Eeckhout, L., 2007. Statistically rigorous Java perfor-
466 mance evaluation, in: Proceedings of the 22nd Annual ACM SIGPLAN Confer-
467 ence on Object-Oriented Programming Systems, Languages and Applications
468 (OOPSLA '07). doi:10.1145/1297027.1297033.
- 469 GitHub Advisory Database, 2015. Verification bypass in jsonwebtoken (CVE-2015-
470 9235, GHSA-c7hr-j4mj-j2w6). [https://github.com/advisories/GHSA-c7h](https://github.com/advisories/GHSA-c7hr-j4mj-j2w6)
471 [r-j4mj-j2w6](https://github.com/advisories/GHSA-c7hr-j4mj-j2w6). Accessed 2026-09-28.

472 Jin, G., Song, L., Shi, X., Scherpelz, J., Lu, S., 2012. Understanding and detecting
473 real-world performance bugs, in: Proceedings of the 33rd ACM SIGPLAN
474 Conference on Programming Language Design and Implementation (PLDI '12),
475 pp. 77–88. doi:10.1145/2254064.2254075.

476 Jones, M.B., Bradley, J., Sakimura, N., 2015. JSON Web Token (JWT). Request for
477 Comments RFC 7519. Internet Engineering Task Force. doi:10.17487/RFC7519.

478 Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D.M., Damian, D.,
479 2014. The promises and perils of mining GitHub, in: Proceedings of the 11th
480 Working Conference on Mining Software Repositories (MSR 2014). doi:10.114
481 5/2597073.2597074.

482 Kaltenecker, C., Mühlbauer, S., Grebhahn, A., Siegmund, N., Apel, S., 2023.
483 Performance evolution of configurable software systems: An empirical study.
484 Empirical Software Engineering 28. doi:10.1007/s10664-023-10338-3.

485 Katha, J.F., Prova, T.T., Morol, M.K., Liew, T.H., Nandi, D., 2026. Replica-
486 tion package: a pre-authentication key parse dominates JWT validation in
487 jsonwebtoken. doi:10.5281/zenodo.23011153. Source repository: [https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominat](https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing-)
488 [es-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing-](https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing-).
489

490 Krüger, S., Späth, J., Ali, K., Bodden, E., Mezini, M., 2018. CrySL: An extensi-
491 ble approach to validating the correct usage of cryptographic APIs, in: 32nd
492 European Conference on Object-Oriented Programming (ECOOP 2018), pp.
493 10:1–10:27. doi:10.4230/LIPIcs.ECOOP.2018.10.

494 Laaber, C., Scheuner, J., Leitner, P., 2019. Software microbenchmarking in the
495 cloud. how bad is it really? Empirical Software Engineering 24. doi:10.1007/s1
496 0664-019-09681-1.

497 Lazar, D., Chen, H., Wang, X., Zeldovich, N., 2014. Why does cryptographic
498 software fail? A case study and open problems, in: Proceedings of the 5th Asia-
499 Pacific Workshop on Systems (APSys '14). doi:10.1145/2637166.2637237.

500 Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2009. Producing wrong
501 data without doing anything obviously wrong!, in: Proceedings of the 14th
502 International Conference on Architectural Support for Programming Languages
503 and Operating Systems (ASPLOS '09). doi:10.1145/1508244.1508275.

504 Nadi, S., Krüger, S., Mezini, M., Bodden, E., 2016. Jumping through hoops, in:
505 Proceedings of the 38th International Conference on Software Engineering (ICSE
506 '16). doi:10.1145/2884781.2884790.

507 Naser, A., Sadeghi Lahijani, M., Wu, C., Gavahi, M., Hoang, V.T., Wang, Z., Yuan,
508 X., 2020. Performance evaluation and modeling of cryptographic libraries for
509 MPI communications. arXiv preprint arXiv:2010.06139.

510 Rahaman, S., Xiao, Y., Afrose, S., Shaon, F., Tian, K., Frantz, M., Kantarcioglu,

511 M., Yao, D.D., 2019. CryptoGuard: High precision detection of cryptographic
512 vulnerabilities in massive-sized Java projects, in: Proceedings of the 2019 ACM
513 SIGSAC Conference on Computer and Communications Security (CCS '19), pp.
514 2455–2472. doi:10.1145/3319535.3345659.

515 Selakovic, M., Glaser, T., Pradel, M., 2017. An actionable performance profiler
516 for optimizing the order of evaluations, in: Proceedings of the 26th ACM
517 SIGSOFT International Symposium on Software Testing and Analysis (ISSTA
518 '17). doi:10.1145/3092703.3092716.

519 Selakovic, M., Pradel, M., 2016. Performance issues and optimizations in JavaScript:
520 An empirical study, in: Proceedings of the 38th International Conference on
521 Software Engineering (ICSE '16), pp. 61–72. doi:10.1145/2884781.2884829.

522 Sheffer, Y., Hardt, D., Jones, M.B., 2020. JSON Web Token Best Current Practices.
523 Request for Comments RFC 8725. Internet Engineering Task Force. doi:10.174
524 87/RFC8725.

525 Sikeridis, D., Kampanakis, P., Devetsikiotis, M., 2020. Assessing the overhead of
526 post-quantum cryptography in TLS 1.3 and SSH, in: Proceedings of the 16th
527 International Conference on Emerging Networking EXperiments and Technologies
528 (CoNEXT '20). doi:10.1145/3386367.3431305.

529 Xu, B., Jia, S., Lin, J., Zheng, F., Ma, Y., Liu, L., Gu, X., Song, L., 2023.
530 JWTKey: Automatic cryptographic vulnerability detection in JWT applications,
531 in: Computer Security – ESORICS 2023, Springer. doi:10.1007/978-3-031-5
532 1479-1_14.

533 Yang, J., Wang, E., Chen, J., Wang, Q., Zhang, Y., Duan, H., Xie, W., Wang,
534 B., 2026. Token time bomb: Evaluating JWT implementations for vulnerability
535 discovery, in: Proceedings of the Network and Distributed System Security
536 Symposium (NDSS '26).

537 Zerouali, A., Mens, T., Robles, G., Gonzalez-Barahona, J.M., 2019. On the
538 relation between outdated Docker containers, severity vulnerabilities, and bugs,
539 in: Proceedings of the 26th IEEE International Conference on Software Analysis,
540 Evolution and Reengineering (SANER '19), pp. 491–501. doi:10.1109/SANER.
541 2019.8668013.

542 Zhu, X., She, G., Xue, B., Zhang, Y., Zhang, Y., Zou, X.K., Duan, X., He, P.,
543 Krishnamurthy, A., Lentz, M., Zhuo, D., Mahajan, R., 2023. Dissecting overheads
544 of service mesh sidecars, in: Proceedings of the 2023 ACM Symposium on Cloud
545 Computing (SoCC '23). doi:10.1145/3620678.3624652.